

Chapter 7 (Scheduling: Introduction) — Homework (Simulation)

Zhiyuan Pan

All runs use `scheduler.py` in `ostep-homework/cpu-sched/`, for example `./scheduler.py -p SJF -l 100,200,300 -c`. In this simulator every job arrives at time 0, so:

- **Response time** = time the job first runs – 0
- **Turnaround time** = time the job completes – 0

Q1

Compute the response time and turnaround time when running three jobs of length 200 with the SJF and FIFO schedulers.

All jobs have the same length, so SJF has nothing to reorder and picks them in arrival order, exactly like FIFO. Both run job 0 during $[0, 200)$, job 1 during $[200, 400)$, and job 2 during $[400, 600)$.

job	response	turnaround
0	0	200
1	200	400
2	400	600
average	200	400

Identical for FIFO and SJF (`-p FIFO -l 200,200,200` and `-p SJF -l 200,200,200`).

Q2

Now do the same but with jobs of different lengths: 100, 200, and 300.

With `-l 100,200,300` the jobs already arrive shortest-first, so SJF again chooses the same order as FIFO: job 0 during [0, 100), job 1 during [100, 300), job 2 during [300, 600).

job (length)	response	turnaround
0 (100)	0	100
1 (200)	100	300
2 (300)	300	600
average	133.33	333.33

Identical for FIFO and SJF.

For contrast, if the jobs arrive in the opposite order (`-l 300,200,100`), FIFO runs the long job first: average response 266.67, average turnaround 466.67. SJF still reorders them shortest-first and gets 133.33 / 333.33. This is the convoy effect that SJF is designed to avoid.

Q3

Now do the same, but also with the RR scheduler and a time-slice of 1.

With `-q 1`, RR switches jobs every time unit, so each job gets the CPU almost immediately.

Three jobs of length 200 (`-p RR -q 1 -l 200,200,200`):

job	response	turnaround
0	0	598
1	1	599
2	2	600

average	1	599
---------	---	-----

Jobs of length 100, 200, 300 (-p RR -q 1 -l 100,200,300):

job (length)	response	turnaround
0 (100)	0	298
1 (200)	1	499
2 (300)	2	600
average	1	465.67

In the second workload, all three jobs share the CPU until job 0 finishes its 100th slice at time 298. Then jobs 1 and 2 alternate until job 1 finishes at 499, and job 2 runs alone until 600.

Comparison of averages:

workload	policy	avg response	avg turnaround
200, 200, 200	FIFO / SJF	200	400
200, 200, 200	RR (q=1)	1	599
100, 200, 300	FIFO / SJF	133.33	333.33
100, 200, 300	RR (q=1)	1	465.67

RR gives excellent response time but the worst turnaround time, because it stretches every job out until nearly the end. SJF is optimal for turnaround but bad for response time. This is the trade-off between fairness and turnaround that the chapter describes.

Q4

For what types of workloads does SJF deliver the same turnaround times as FIFO?

SJF gives the same turnaround as FIFO **whenever FIFO already happens to run the jobs shortest-first**, i.e. when the jobs arrive in order of non-decreasing length. Special cases:

- **All jobs have the same length** (Q1: both 400).
- **Jobs arrive already sorted from shortest to longest** (Q2, -l 100,200,300 : both 333.33).

If a longer job arrives before a shorter one, SJF moves the short job forward and FIFO's turnaround is worse (-l 300,200,100 : FIFO 466.67 vs SJF 333.33).

Q5

For what types of workloads and quantum lengths does SJF deliver the same response times as RR?

RR gives the same response times as SJF when RR ends up running the jobs one after another in shortest-first order. That needs two conditions:

1. **The jobs arrive in shortest-first order** (so RR's order equals SJF's), and
2. **The quantum is at least as long as every job except the last one**, so each job runs to completion within its first time slice and RR never actually preempts anyone before the last job starts.

Evidence with -l 100,200,300 (SJF average response = 133.33):

RR quantum	avg response	same as SJF?
1	1	no
100	100	no (job 1 is preempted after 100, so job 2 starts at 200)
200	133.33	yes ($q \geq 100$ and $q \geq 200$)
300	133.33	yes

Likewise, three jobs of 200 with $q \geq 200$ give 200 for both. If the jobs arrive longest-first, no quantum works: -p RR -q 300 -l 300,200,100 gives 266.67, while SJF gives 133.33.

Q6

What happens to response time with SJF as job lengths increase? Can you use the simulator to demonstrate the trend?

Under SJF, a job's response time is the **sum of the lengths of all the shorter jobs that run before it**. So response time grows **linearly** with job length: double every job's length and the response time doubles.

Three equal jobs, -p SJF -l L,L,L :

job length L	avg response
10	10
100	100
200	200
500	500
1000	1000

Scaling a mixed workload, -p SJF -l L,2L,3L :

jobs	avg response
100, 200, 300	133.33
200, 400, 600	266.67
400, 800, 1200	533.33

For N equal jobs of length L, the average response time is $L \cdot (N-1)/2$, so it grows with both the job length and the number of jobs. SJF never interrupts a running job, so every job waits for all the jobs ahead of it to finish completely. When jobs are long, that wait is long. This is why SJF is bad for interactive work.

Q7

What happens to response time with RR as quantum lengths increase? Can you write an equation that gives the worst-case response time, given N jobs?

Response time with RR **increases linearly with the quantum**. The k -th job in the queue (counting from 0) has to wait for each of the k jobs ahead of it to use one time slice.

Five jobs of length 200, -p RR -q Q -l 200,200,200,200,200 :

quantum Q	response of last job	avg response
1	4	2
10	40	20
50	200	100
100	400	200
200	800	400

For example, with $Q = 10$ the responses are 0, 10, 20, 30, 40. When Q reaches the job length (200), RR turns into FIFO.

Worst-case response time for N jobs (all arriving at time 0), with quantum q :

$$T_{\text{response, worst}} = (N - 1) \cdot q$$

The last job in the queue waits for the other $N - 1$ jobs to each use one full time slice. More precisely, job i has response time

$\sum_{j < i} \min(q, \text{len}_j)$. This is at most $(N - 1)q$, with equality when every job ahead of it is at least q long. The **average** response time in that case is $\frac{(N-1)q}{2}$.

The formula shows the RR trade-off: a small quantum gives good response time, but more frequent context switches, whose cost (measured in chapter 6 at about $1 \mu\text{s}$ each) has to be amortized. A large quantum amortizes switching better, but response time grows as $(N - 1) \cdot q$.